

**Project Report**  
**On**  
**eAuto – Bike Spare Parts E-Commerce Platform PHP**  
**In partial fulfillment for the award of the degree Of**  
**BACHELOR OF COMPUTER APPLICATION**  
**[B.C.A]**  
**Year 2025-2026**

**SUBMITTED BY:**  
**Dhruv Dholiya (4924)**  
**Dharmik Dobariya (4928)**  
**Vatsal Kapadia (5029)**

**GUIDED BY:**  
**Prof. Yesha Patel**

**BCA-5th SEMESTER**

**Submitted to:**



**SHRI SHAMBHUBHAI V. PATEL COLLEGE OF**  
**COMPUTER SCIENCE & BUSINESS**

**MANAGEMENT**

**Affiliated to**

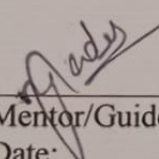
**Veer Narmad South Gujarat University**

SHRI SHAMBHUBHAI V. PATEL COLLEGE OF  
COMPUTER SCIENCE & BUSINESS MANAGEMENT  
Near I. C. Gandhi School, Sumul Dairy Road, Surat (Gujarat), India



## CERTIFICATE

This is to certify that the project entitled eAuto - Bike Spare Parts  
E-commerce platform in Subject Advance Web Designing  
has been carried out by Mr. / Ms. Dholiya Dhruv Bhaveshbhai  
in Bachelor of Computer Application, Semester V, Exam Seat No. 4924  
towards partial fulfillment of the course, for the Academic Year 2025-26.

  
Mentor/Guide  
Date:



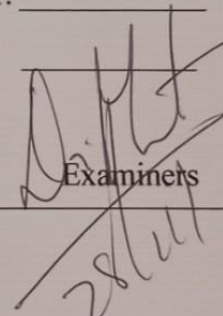
  
I/c Principal  
Date:



PROJECT OF B.C.A.

Academic Year: \_\_\_\_\_

Approved by: \_\_\_\_\_

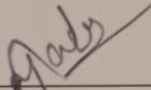
  
Examiners

SHRI SHAMBHUBHAI V. PATEL COLLEGE OF  
COMPUTER SCIENCE & BUSINESS MANAGEMENT  
Near I. C. Gandhi School, Sumul Dairy Road, Surat (Gujarat), India



## CERTIFICATE

This is to certify that the project entitled Bike spare parts E -  
Commerce Platform in Subject Advance Web Designing  
has been carried out by Mr. / Ms. Dharmik Sanjaybhai Dobariya  
in Bachelor of Computer Application, Semester V, Exam Seat No. 4928  
towards partial fulfillment of the course, for the Academic Year 2025-26.

  
Mentor/Guide  
Date:



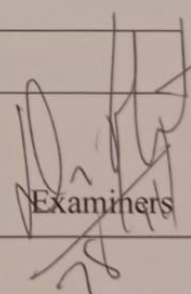
  
I/c Principal  
Date:



PROJECT OF B.C.A.

Academic Year: \_\_\_\_\_

Approved by: \_\_\_\_\_

  
Examiners

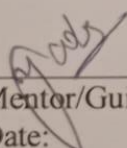


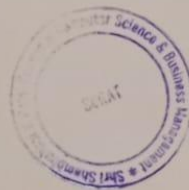
**SHRI SHAMBHUBHAI V. PATEL COLLEGE OF  
COMPUTER SCIENCE & BUSINESS MANAGEMENT**  
Near I. C. Gandhi School, Sumul Dairy Road, Surat (Gujarat), India

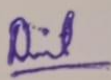


## **CERTIFICATE**

*This is to certify that the project entitled Bike Spare Parts E-commerce Platform in Subject Advance Web Designing has been carried out by Mr. / Ms. Kapodiya Kateal Vipul Kumar in Bachelor of Computer Application, Semester V, Exam Seat No. 6029 towards partial fulfillment of the course, for the Academic Year 2025-26.*

  
Mentor/Guide  
Date:



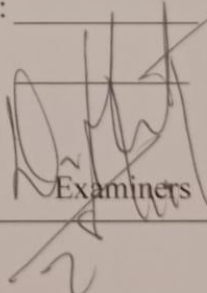
  
I/c Principal  
Date:



**PROJECT OF B.C.A.**

Academic Year: \_\_\_\_\_

Approved by: \_\_\_\_\_

  
Examiners

## **Acknowledgment**

We would like to take this opportunity to express our heartfelt gratitude to everyone who contributed to the successful completion of our project, **“eAuto – Bike Spare Parts E-Commerce Platform.”**

We are especially grateful to our project guide, **Prof. Yesha Patel**, for her invaluable guidance, continuous encouragement, and insightful feedback throughout the course of this work. Her expert advice and constant motivation have been a source of inspiration and have greatly helped us in achieving our objectives.

We also wish to extend our thanks to all the faculty members of our department for their cooperation, constructive suggestions, and for providing us with the necessary resources and knowledge that supported our project development.

Our sincere appreciation goes to our friends and classmates, whose timely help and support provided us with motivation and confidence during various phases of this project.

Lastly, we would like to acknowledge the dedication, teamwork, and mutual support of our project members, without whom this work would not have been possible:

- **Dhruv Dholiya (4924)**
- **Dharmik Dobariya (4728)**
- **Vatsal Kapadia (5029)**

Together, we have shared valuable experiences that will remain with us beyond the completion of this project.

---

# Index

| SR.NO     | PARTICULAR                                  | PAGE NO |
|-----------|---------------------------------------------|---------|
| <b>1</b>  | <b>Introduction</b>                         |         |
|           | 1.1 Background of the Project               |         |
|           | 1.2 Purpose of the Project                  |         |
|           | 1.3 Scope of the Project                    |         |
|           | 1.4 Objectives                              |         |
|           | 1.5 Advantages of the System                |         |
| <b>2</b>  | <b>Aim of the Project</b>                   |         |
|           | 2.1 Statement of Aim                        |         |
|           | 2.2 Importance of Aim                       |         |
|           | 2.3 How Aim Solves Current Problems         |         |
| <b>3</b>  | <b>Features of the Project</b>              |         |
|           | 3.1 User-side Features                      |         |
|           | 3.1.1 User Registration & Login             |         |
|           | 3.1.2 Browse & Search Products              |         |
|           | 3.1.3 Cart & Checkout System                |         |
|           | 3.1.4 Order Tracking                        |         |
|           | 3.1.5 Profile & Delivery Address Management |         |
|           | 3.2 Admin-side Features                     |         |
|           | 3.2.1 Admin Login & Dashboard               |         |
|           | 3.2.2 Manage Products (Add/Edit/Delete)     |         |
|           | 3.2.3 Manage Orders                         |         |
|           | 3.2.4 Manage Users                          |         |
|           | 3.2.5 Generate Reports                      |         |
| <b>4</b>  | <b>System Analysis &amp; Design</b>         |         |
|           | 4.1 Requirement Gathering                   |         |
|           | 4.2 Functional Requirements                 |         |
|           | 4.3 Non-Functional Requirements             |         |
|           | 4.4 Technology Stack                        |         |
|           | 4.5 Database Design                         |         |
| <b>5.</b> | <b>E-R Diagram</b>                          |         |
|           | 5.1 Entities                                |         |
|           | 5.2 Relationships                           |         |
|           | 5.3 Attributes                              |         |
|           | 5.4 Final ER Model                          |         |
| <b>6.</b> | <b>Data Flow Diagram (DFD)</b>              |         |
|           | 6.1 Context Level DFD (Level 0)             |         |
|           | 6.2 Level 1 DFD – User Processes            |         |
|           | 1.0 Login & Registration                    |         |

|           |                                                                                   |  |
|-----------|-----------------------------------------------------------------------------------|--|
|           | 2.0 Product Management                                                            |  |
|           | 3.0 Cart & Order Management                                                       |  |
|           | 4.0 Payment Processing                                                            |  |
|           | 5.0 Supplier & Inventory Management                                               |  |
|           | 6.0 Delivery Management                                                           |  |
|           | 7.0 Reports & Analytics                                                           |  |
|           | 6.3 Explanation of Data Stores (MongoDB Collections)                              |  |
| <b>7</b>  | <b>Flow Chart</b>                                                                 |  |
|           | 7.1 User Registration & Login Flow                                                |  |
|           | 7.2 Product Purchase Flow                                                         |  |
|           | 7.3 Order Management Flow                                                         |  |
|           | 7.4 Payment Processing Flow                                                       |  |
| <b>8</b>  | <b>Implementation Details</b>                                                     |  |
|           | 8.1 Frontend (React JS, Bootstrap, AOS)                                           |  |
|           | 8.2 Backend (PHP – Core PHP for APIs & Logic)                                     |  |
|           | 8.3 Database (MongoDB Compass – Collections & Schema)                             |  |
|           | 8.4 APIs & Endpoints (PHP → React communication with JSON)                        |  |
|           | 8.5 Security Features (Authentication, Session Management, Validation)            |  |
| <b>9</b>  | <b>Testing</b>                                                                    |  |
|           | 9.1 Test Cases (Unit & Integration)                                               |  |
|           | 9.2 User Testing (UI/UX & Functional)                                             |  |
|           | 9.3 Admin Testing                                                                 |  |
|           | 9.4 Performance & Load Testing                                                    |  |
| <b>10</b> | <b>Limitations of the Project</b>                                                 |  |
|           | 10.1 Technical Limitations (React-PHP integration, MongoDB structure)             |  |
|           | 10.2 Functional Limitations (Limited Payment Methods, Delivery System)            |  |
|           | 10.3 Future Enhancements (Mobile App, AI-based Recommendations, Advanced Reports) |  |
| <b>11</b> | <b>Conclusion</b>                                                                 |  |
|           | 11.1 Summary of Work                                                              |  |
|           | 11.2 Achievements                                                                 |  |
|           | 11.3 Future Scope                                                                 |  |
| <b>12</b> | <b>References</b>                                                                 |  |



# **1. Introduction**

## **1.1 Background of the Project**

With the rapid growth of e-commerce, traditional methods of purchasing automobile spare parts are becoming outdated. Customers often face difficulties such as limited availability of parts, lack of price transparency, and difficulty in finding reliable vendors. The eAuto platform aims to address these issues by providing a centralized online marketplace for bike spare parts, offering convenience, affordability, and accessibility.

---

## **1.2 Purpose of the Project**

The primary purpose of this project is to build an online platform where users can easily browse, compare, and purchase bike spare parts. The system benefits both customers (by offering a wide range of products and hassle-free shopping) and administrators (by providing tools to manage products, users, orders, and deliveries efficiently).

---

## **1.3 Scope of the Project**

The scope of the eAuto project includes:

- User-side features such as product browsing, cart management, checkout, payment integration, order tracking, and user profile management.

- Admin-side features including product management, user management, order tracking, delivery assignment, and report generation.
  - Database integration to maintain records of products, users, orders, and payments.
  - Responsive front-end design for mobile and desktop users.
- 

## 1.4 Objectives

The main objectives of the project are:

- To provide a user-friendly platform for purchasing bike spare parts online.
  - To reduce the gap between suppliers and customers through a centralized system.
  - To ensure secure payment processing and accurate order tracking.
  - To enable administrators to manage inventory, users, and reports efficiently.
  - To support future scalability by adding more brands, products, and features.
- 

## 1.5 Advantages of the System

The proposed system offers several benefits, such as:

- **Convenience:** Customers can order parts anytime, anywhere.
- **Variety:** Wide range of spare parts from multiple brands in one place.

- **Transparency:** Clear product details, pricing, and discounts.
  - **Efficiency:** Faster order processing and delivery management.
  - **Data-driven:** Sales and user data help admins make better business decisions.
  - **Scalability:** System can easily integrate with additional brands, suppliers, or services in the future.
-

## **2. Aim of the Project**

### **2.1 Statement of Aim**

The aim of the eAuto project is to design and develop a comprehensive **online e-commerce platform** for bike spare parts that allows users to easily search, compare, and purchase products, while providing administrators with effective tools for managing products, users, orders, and delivery processes.

---

### **2.2 Importance of Aim**

The importance of this aim lies in addressing the major challenges faced in the traditional spare parts market:

- **Accessibility:** Customers often struggle to find genuine spare parts at local shops.
  - **Time Saving:** Instead of physically visiting multiple stores, users can buy parts online with just a few clicks.
  - **Reliability:** Ensures quality assurance through verified sellers and transparent pricing.
  - **Business Growth:** Helps suppliers and administrators expand their reach to a larger customer base.
  - **Technological Advancement:** Shifts the spare parts industry toward digital transformation, aligning with modern consumer habits.
-

## 2.3 How Aim Solves Current Problems

The aim of this project provides solutions to the existing problems in the bike spare parts domain by:

- **Centralized Platform:** Eliminates the need for customers to visit multiple vendors by offering all spare parts in one place.
  - **Real-time Availability:** Provides up-to-date stock and pricing information.
  - **Secure Transactions:** Integrates reliable payment gateways to ensure safe purchases.
  - **Order Tracking:** Customers can track the status of their orders and deliveries online.
  - **Efficient Management:** Admin panel allows streamlined management of inventory, users, and reports.
  - **Customer Satisfaction:** Reduces hassle, improves transparency, and enhances the overall buying experience.
-

## **3. Features of the Project**

### **3.1 User-side Features**

#### **3.1.1 User Registration & Login**

- New users can create an account by registering with their email, phone number, and password.
  - Existing users can log in using secure authentication.
  - Sessions are maintained for user activities like browsing, cart, and checkout.
- 

#### **3.1.2 Browse & Search Products**

- Users can browse spare parts based on **brand (Hero, Honda, Bajaj, etc.)**, categories, or price range.
  - A **search bar** enables quick product lookup by name or part number.
  - Product details such as image, description, price, discounts, and availability are displayed.
- 

#### **3.1.3 Cart & Checkout System**

- Users can add multiple products to their **shopping cart**.
- Cart preview is available in the header for easy access.

- Checkout includes selection of **delivery address, payment method (Cash, Card, UPI, NetBanking)**, and order confirmation.
- 

#### 3.1.4 Order Tracking

- Customers can view their **order history** and the **current status** (Pending, Processed, Shipped, Delivered).
  - Real-time updates ensure transparency in the delivery process.
- 

#### 3.1.5 Profile & Delivery Address Management

- Users can manage their **personal details, password, and contact information**.
  - Multiple delivery addresses can be added, edited, or deleted for convenience.
- 

### 3.2 Admin-side Features

#### 3.2.1 Admin Login & Dashboard

- Secure admin login with authentication.
  - Dashboard displays **key statistics** like total users, orders, sales, and product inventory status.
- 

#### 3.2.2 Manage Products (Add/Edit/Delete)



- Admins can **add new products** with details like name, brand, price, discount, stock quantity, and image.
  - Existing products can be **edited or updated**.
  - Outdated or unavailable products can be **deleted** from the system.
- 

### 3.2.3 Manage Orders

- Admins can view all orders placed by users.
  - Orders can be updated with statuses such as **Pending, Approved, Shipped, Delivered, or Cancelled**.
  - Delivery staff can be assigned to orders for timely dispatch.
- 

### 3.2.4 Manage Users

- Admins can view and manage registered users.
  - Option to **block or remove users** in case of fraudulent activity.
  - User data (profile, order history) can be accessed for support purposes.
- 

### 3.2.5 Generate Reports

- Reports can be generated for **sales, revenue, product performance, and customer activity**.
  - Helps administrators analyze business growth and make data-driven decisions.
-

## **4. System Analysis & Design**

### **4.1 Requirement Gathering**

Requirement gathering is done by analyzing the needs of customers and administrators. Sources include surveys, existing system analysis, and direct feedback.

- **User Requirements:**
    - Easy product browsing and searching.
    - Secure login and profile management.
    - Cart, checkout, and payment options.
    - Order tracking and history.
  - **Admin Requirements:**
    - Ability to manage products, users, and orders.
    - Generate reports for sales and performance.
    - Dashboard for real-time insights.
- 

### **4.2 Functional Requirements**

Functional requirements describe what the system should do:

1. **User-side**
  - Register/Login/Logout functionality.
  - Browse products by category/brand.
  - Search products with filters.

- Add to cart, view cart, and checkout.
- Manage profile and delivery address.
- Track orders and view history.

## 2. Admin-side

- Secure login for admins.
- Manage product details (add, update, delete).
- Manage users and their activities.
- Manage orders (status updates, assign delivery).
- Generate sales and inventory reports.

---

### 4.3 Non-Functional Requirements

These define system qualities rather than features:

- **Performance:** Fast loading times, responsive design, optimized queries.
- **Scalability:** Support for future expansion with more users and products.
- **Security:** Encrypted passwords, secure payment gateway, session management.
- **Usability:** Simple and intuitive UI for both users and admins.
- **Availability:** High uptime with reliable hosting.
- **Maintainability:** Clean code structure, modular design, and easy updates.

---

### 4.4 Technology Stack

#### Front-End

- **React.js** → Component-based architecture for building dynamic and reusable UI.
- **HTML5** → Structure of web pages.
- **CSS3** → Styling, animations, and responsive layouts.
- **JavaScript (ES6+)** → Client-side interactivity and logic.
- **Bootstrap 5** → Predefined responsive UI components.
- **AOS (Animate On Scroll)** → Smooth scroll-based animations.
- **PHP (for legacy UI/session handling)** → Used for rendering certain UI components and managing sessions (in earlier implementation).

## **Back-End**

- **Option 1 (Earlier Stack – PHP)**
  - **Core PHP** → Server-side scripting and session handling.
  - **XAMPP (Apache + PHP + MySQL)** → Local development server.
- **Option 2 (Modern Stack – Node.js)**
  - **Node.js with Express.js** → REST API development and backend logic.
  - **Integration with MySQL** → For managing product, user, and order data.

## **Database**

- **MySQL** → Primary relational database for structured data (Users, Products, Orders, Cart, Reports).

- **MongoDB Compass (Optional)** → GUI tool for managing MongoDB if a hybrid/noSQL database is considered for future scalability.

## Tools & Platforms

- **XAMPP / Node Server** → For local testing and development.
- **Git & GitHub** → Version control and collaborative development.
- **MongoDB Compass** → Database GUI for NoSQL database management (if used).

## Payment Integration

- **Razorpay / PayPal / Stripe APIs** → Secure and seamless online payments via multiple modes (Cards, UPI, NetBanking, Wallets).

---

## 4.5 Database Design

### 1. Admins Collection

```
{  
  
  "_id": ObjectId("..."),  
  
  "email": "admin@eauto.com",  
  
  "password": "hashed_password",  
  
  "createdAt": ISODate("2025-09-03T10:00:00Z")  
}
```

---

## 2. Users Collection

```
{
  "_id": ObjectId("..."),
  "fullname": "Rahul Sharma",
  "email": "rahul@example.com",
  "contact": "+91-9876543210",
  "birthdate": ISODate("1995-05-10"),
  "gender": "Male",
  "password": "hashed_password",
  "addresses": [
    {
      "addressId": ObjectId("..."),
      "line1": "123 Street",
      "city": "Surat",
      "state": "Gujarat",
      "zip": "395001",
      "isDefault": true
    }
  ]
}
```

---

### 3. Products Collection

```
{  
  "_id": ObjectId("..."),  
  "productName": "Hero Splendor Brake Pad",  
  "price": 750,  
  "originalPrice": 900,  
  "discountAmount": 150,  
  "image": "https://cdn.eauto.com/products/brakepad1.jpg",  
  "brand": "Splendor",  
  "mainBrand": "Hero",  
  "rating": 4.3,  
  "reviews": 58,  
  "stockStatus": "In Stock"  
}
```

---

### 4. Cart Collection

```
{  
  "_id": ObjectId("..."),  
  "userId": ObjectId("..."),  
  "items": [  

```

```
{  
  "productId": ObjectId("..."),  
  "productName": "Hero Splendor Brake Pad",  
  "price": 750,  
  "image": "https://cdn.eauto.com/products/brakepad1.jpg",  
  "brand": "Splendor",  
  "quantity": 2,  
  "subtotal": 1500  
}  
],  
"updatedAt": ISODate("2025-09-03T11:30:00Z")  
}
```

---

## 5. Orders Collection

```
{  
  "_id": ObjectId("..."),  
  "userId": ObjectId("..."),  
  "fullname": "Rahul Sharma",  
  "email": "rahul@example.com",  
  "phone": "9876543210",  
  "address": {
```



```
"line1": "123 Street",  
  
"city": "Surat",  
  
"state": "Gujarat",  
  
"zip": "395001"  
  
},  
  
"totalAmount": 1500,  
  
"paymentMethod": "UPI",  
  
"status": "Pending",  
  
"createdAt": ISODate("2025-09-03T11:45:00Z"),  
  
"items": [  
  
  {  
  
    "productId": ObjectId("..."),  
  
    "productName": "Hero Splendor Brake Pad",  
  
    "quantity": 2,  
  
    "price": 750,  
  
    "subtotal": 1500  
  
  }  
  
]  
  
}
```

## Final MongoDB Collections

- admins
  - users
  - products
  - carts
  - orders
-

## **5. E–R Diagram**

### **5.1 Entities**

From the database design, the main entities are:

1. **Admins**
  2. **Users**
  3. **Products**
  4. **Cart**
  5. **Orders**
  6. **Order\_Items**
  7. **Delivery Address (inside Orders table)**
- 

### **5.2 Relationships**

1. **User → Orders:**
  - One user can place many orders.
  - Relationship: **1:N**
2. **Orders → Order\_Items:**
  - One order can contain many products (items).
  - Relationship: **1:N**
3. **Products → Order\_Items:**
  - One product can appear in many orders.
  - Relationship: **1:N**

#### 4. **User → Cart:**

- One user can maintain many cart items.
- Relationship: **1:N**

#### 5. **Products → Cart:**

- A product can exist in many users' carts.
- Relationship: **1:N**

#### 6. **Admins → Products/Orders/Users:**

- Admins manage products, orders, and users.
  - Relationship: **1:N (management)**
- 

### 5.3 Attributes

#### **Admins**

- id (PK), email, password, created\_at

#### **Users**

- id (PK), fullname, email, contact, birthdate, gender, password

#### **Products**

- id (PK), product\_name, price, original\_price, discount\_amount, image, brand, main\_brand, rating, reviews, stock\_status

#### **Cart**

- id (PK), product\_name, price, image, brand, quantity

#### **Orders**

- id (PK), user\_id (FK), fullname, email, phone, address, city, state, zip, total\_amount, payment\_method, created\_at

## Order\_Items

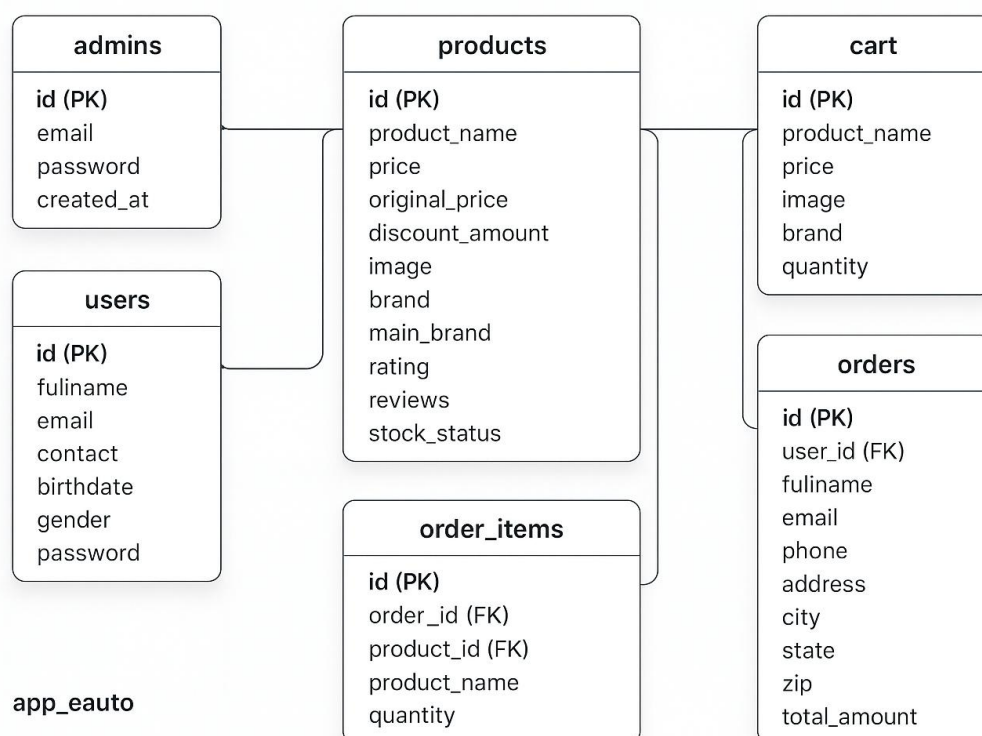
- id (PK), order\_id (FK), product\_id (FK), product\_name, quantity, price

---

## 5.4 Final ER Model

The final ER Model will include:

- **Users** linked to **Cart** and **Orders**
- **Orders** linked to **Order\_Items**
- **Products** linked to **Cart** and **Order\_Items**
- **Admins** linked to **Products**, **Orders**, **Users**



## **6. Data Flow Diagram (DFD)**

### **6.1 Context Level DFD (Level 0)**

#### **Central Process**

#### **eAuto Management System**

---

#### **High-Level Sub-Processes**

1. **Login & Registration** – Handles Customer/Admin authentication
  2. **Product Management** – Search, browse, and manage product catalog
  3. **Cart & Order Management** – Add to cart, checkout, confirm orders
  4. **Payment Processing** – Verify and confirm transactions
  5. **Supplier & Inventory Management** – Stock updates, stock requests
  6. **Delivery Management** – Assign deliveries, track status
  7. **Reports & Analytics** – Generate sales, stock, and delivery reports for Admin
- 

#### **External Entities**

- **Customer**
- **Admin**
- **Supplier**
- **Delivery Staff**

---

## Data Stores

| Data Store                              | Description                                     |
|-----------------------------------------|-------------------------------------------------|
| <b>D1: User Database</b>                | Customer & admin login/registration details     |
| <b>D2: Product Database</b>             | Product details, categories, pricing, discounts |
| <b>D3: Cart &amp; Order Database</b>    | Cart info, order details, order history         |
| <b>D4: Payment/Transaction Database</b> | Payment confirmations, transaction logs         |
| <b>D5: Inventory Database</b>           | Stock levels, supplier updates                  |
| <b>D6: Delivery Database</b>            | Delivery assignments, status updates            |
| <b>D7: Reports Database</b>             | Aggregated data for admin reports/analytics     |

---

## Data Flow (Movement)

### Customer ↔ eAuto System

- **Requests:** Register/Login, Search Products, Add to Cart, Place Order, Payment, View Orders
- **Responses:** Login Confirmation, Search Results, Order Confirmation, Payment Status, Delivery Status
- **Databases Accessed:** D1, D2, D3, D4, D6

### Admin ↔ eAuto System

- **Requests:** Manage Products, Orders, Users; View Reports
- **Responses:** Confirmation Messages, Updated Data, Reports
- **Databases Accessed:** D1, D2, D3, D5, D6, D7

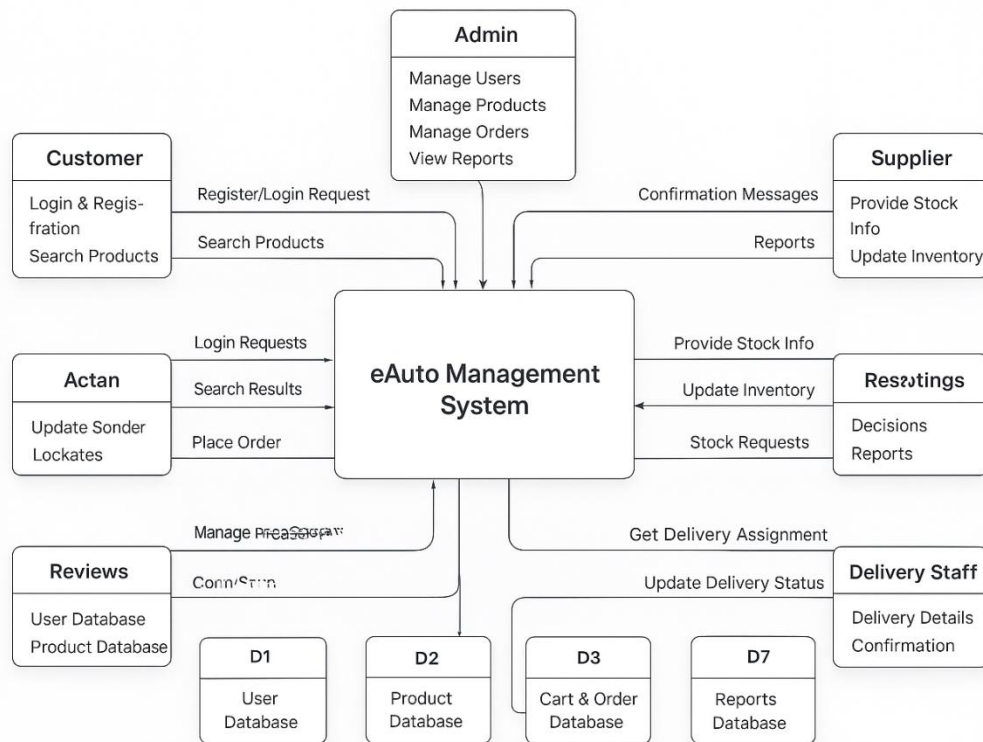
### Supplier ↔ eAuto System

- **Requests:** Provide Stock Info, Update Inventory

- **Responses:** Stock Confirmation, Order Requests
- **Databases Accessed:** D5

### Delivery Staff ↔ eAuto System

- **Requests:** Delivery Assignment, Update Delivery Status
- **Responses:** Delivery Details, Confirmation
- **Databases Accessed:** D3, D6





## 6.2 Level 1 DFD – User Processes

### Processes (Inside the System)

| Process                                        | Input                                                         | Output                               | Data Store                |
|------------------------------------------------|---------------------------------------------------------------|--------------------------------------|---------------------------|
| <b>1.0 Login &amp; Registration</b>            | Login/Register request (Customer/Admin)                       | Login confirmation                   | D1: User Database         |
| <b>2.0 Product Management</b>                  | Product search request (Customer), Add/Update product (Admin) | Product info / Search results        | D2: Product Database      |
| <b>3.0 Cart &amp; Order Management</b>         | Add to Cart, Place Order (Customer)                           | Order confirmation                   | D3: Cart & Order Database |
| <b>4.0 Payment Processing</b>                  | Payment details (Customer)                                    | Transaction confirmation             | D4: Payment Records       |
| <b>5.0 Supplier &amp; Inventory Management</b> | Stock info update (Supplier)                                  | Stock confirmation                   | D5: Inventory Database    |
| <b>6.0 Delivery Management</b>                 | Delivery assignment request (System → Delivery Staff)         | Delivery confirmation, Status update | D6: Delivery Records      |
| <b>7.0 Reports &amp; Analytics</b>             | Report request (Admin)                                        | Sales / Stock / Delivery reports     | D7: Reports Database      |

---

### External Entities

- **Customer** – Register/Login, Search Products, Add to Cart, Make Payment, Track Delivery
  - **Admin** – Manage Products, Orders, Users, Reports
  - **Supplier** – Stock Updates, Inventory Info
  - **Delivery Staff** – Order Delivery, Update Delivery Status
-

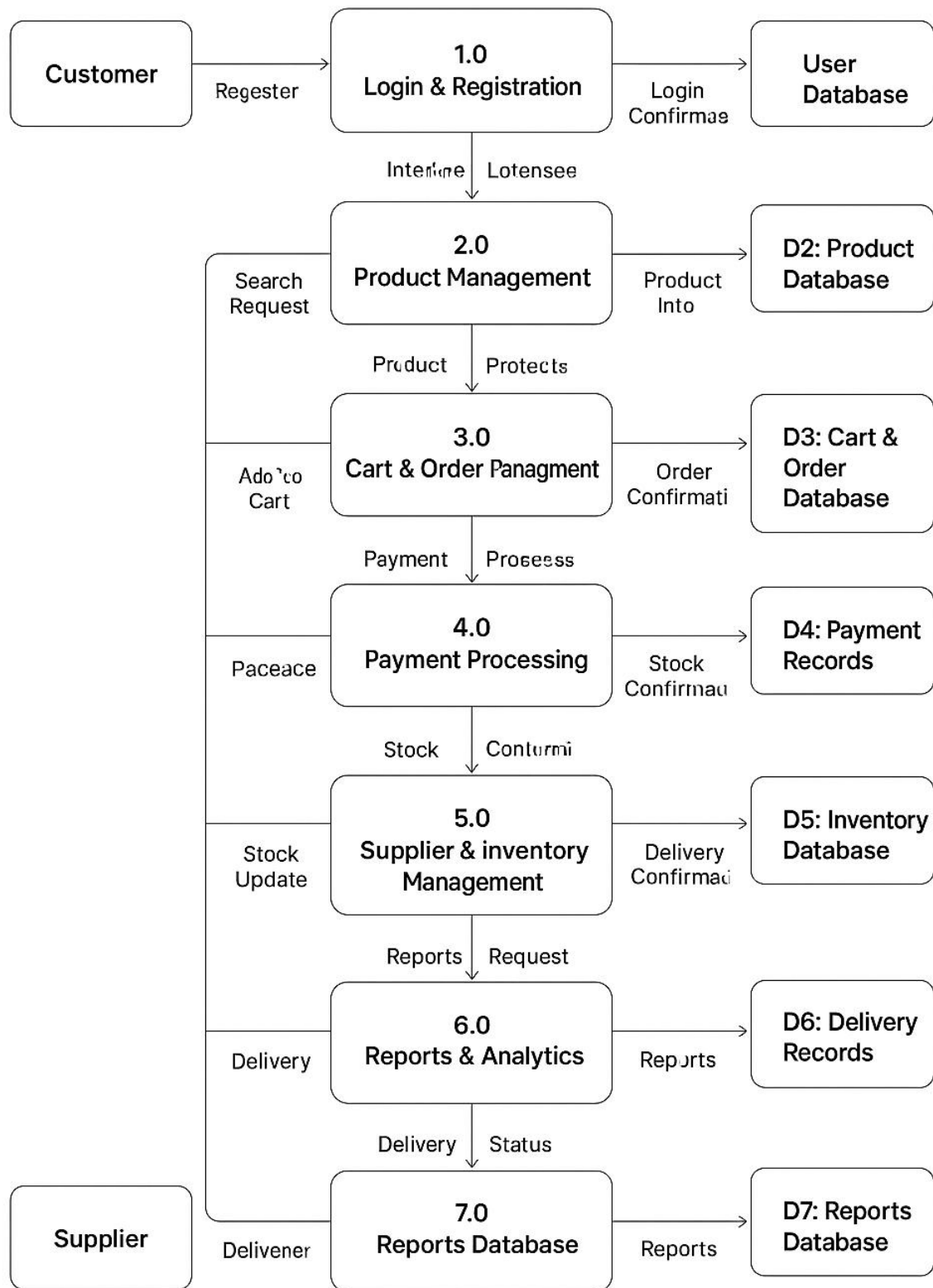
**Data Stores**

| Data Store                | Description                            |
|---------------------------|----------------------------------------|
| D1: User Database         | Stores customer/admin login info       |
| D2: Product Database      | Stores product details                 |
| D3: Cart & Order Database | Stores cart & order transactions       |
| D4: Payment Records       | Stores payment details                 |
| D5: Inventory Database    | Manages stock levels                   |
| D6: Delivery Records      | Stores delivery status and assignments |
| D7: Reports Database      | Stores analytics data                  |

**Data Flows**

| Entity                                              | Process                               | Data Store            |
|-----------------------------------------------------|---------------------------------------|-----------------------|
| Customer ↔ 1.0 Login & Registration ↔ D1            | Authentication / Registration         | User Database         |
| Customer ↔ 2.0 Product Management ↔ D2              | Product search / browsing             | Product Database      |
| Customer ↔ 3.0 Cart & Order Management ↔ D3         | Add to cart, Place order              | Cart & Order Database |
| Customer ↔ 4.0 Payment Processing ↔ D4              | Payment processing                    | Payment Records       |
| Supplier ↔ 5.0 Supplier & Inventory Management ↔ D5 | Stock updates                         | Inventory Database    |
| Delivery Staff ↔ 6.0 Delivery Management ↔ D6       | Delivery assignments / status updates | Delivery Records      |
| Admin ↔ 7.0 Reports & Analytics ↔ D7                | Generate reports                      | Reports Database      |

## eAuto Management System – Level 1 DFD



## 1.0 Login & Registration

### External Entities

- Customer
- Admin

### Data Store

- D1: User Database

### Sub-Processes

#### 1.1 Register User

- **Input:** Registration details (name, email, password, etc.) from Customer/Admin
- **Process:** Validate details, store new record
- **Output:** Registration confirmation
- **Data Store Interaction:** Insert new user into **D1: User Database**

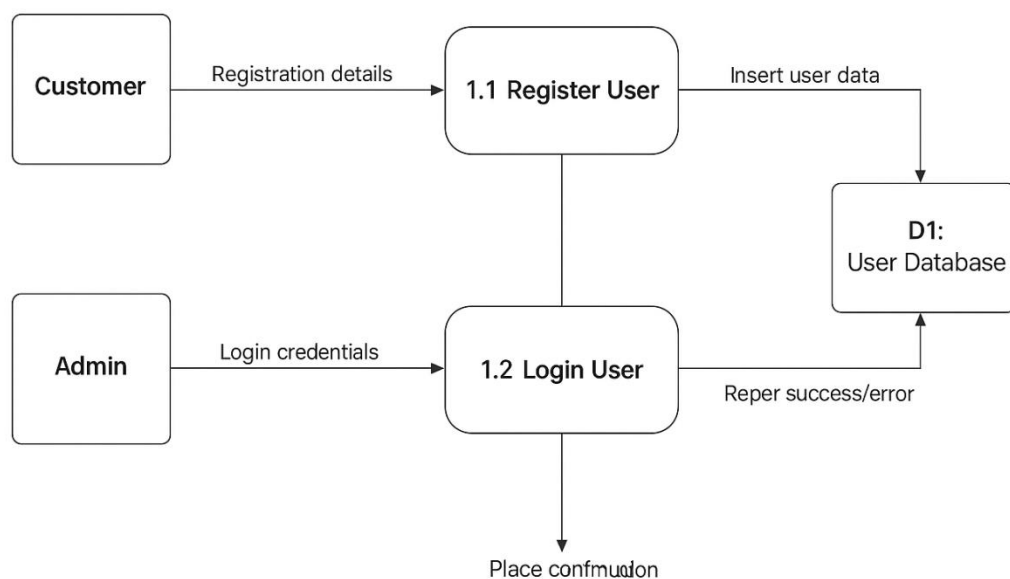
#### 1.2 Login User

- **Input:** Login credentials (username/email & password) from Customer/Admin
  - **Process:** Validate credentials
  - **Output:** Login success/failure message
  - **Data Store Interaction:** Verify credentials from **D1: User Database**
-

## Data Flows

| From                                  | To                         | Data   |
|---------------------------------------|----------------------------|--------|
| Customer/Admin → 1.1 Register User    | Registration details       | Input  |
| 1.1 Register User → D1: User Database | Store data                 | Insert |
| Customer/Admin → 1.2 Login User       | Login credentials          | Input  |
| 1.2 Login User → D1: User Database    | Verify credentials         | Query  |
| 1.1 Register User → Customer/Admin    | Registration confirmation  | Output |
| 1.2 Login User → Customer/Admin       | Login confirmation / error | Output |

## DFD Level-2 – 1.0 Login & Registration – eAuto Management System



## 2.0 Product Management

### External Entities

- Customer
- Admin

### Data Store

- D2: Product Database

### Sub-Processes

#### 2.1 Search Product

- **Input:** Product search request from Customer
- **Process:** Retrieve matching product info
- **Output:** Search results (product list/details)
- **Data Store Interaction:** Read from **D2: Product Database**

#### 2.2 Add Product

- **Input:** New product details from Admin
- **Process:** Validate and insert new product
- **Output:** Product added confirmation
- **Data Store Interaction:** Insert into **D2: Product Database**

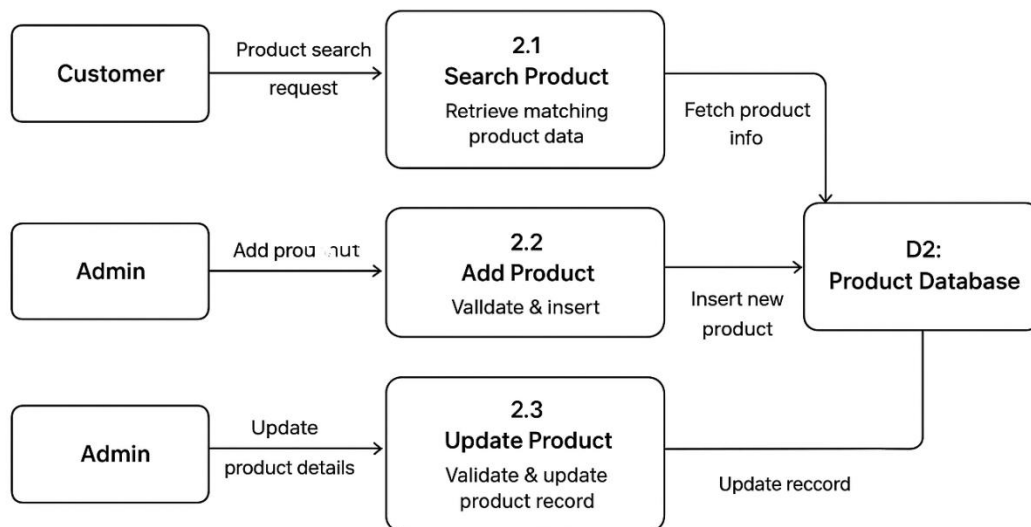
#### 2.3 Update Product

- **Input:** Updated product details from Admin
- **Process:** Validate and update product record
- **Output:** Update confirmation
- **Data Store Interaction:** Update record in **D2: Product Database**

## Data Flows

| From                                      | To                      | Data     |
|-------------------------------------------|-------------------------|----------|
| Customer → 2.1 Search Product             | Product search request  | Input    |
| 2.1 Search Product → D2: Product Database | Fetch product info      | Query    |
| D2: Product Database → 2.1 Search Product | Product details         | Response |
| 2.1 Search Product → Customer             | Search results          | Output   |
| Admin → 2.2 Add Product                   | New product details     | Input    |
| 2.2 Add Product → D2: Product Database    | Insert new product      | Insert   |
| 2.2 Add Product → Admin                   | Add confirmation        | Output   |
| Admin → 2.3 Update Product                | Updated product details | Input    |
| 2.3 Update Product → D2: Product Database | Update record           | Update   |
| 2.3 Update Product → Admin                | Update confirmation     | Output   |

### 2.0 Product Management – eAuto Management System



## 3.0 Cart & Order Management

### External Entities

- Customer

### Data Store

- D3: Cart & Order Database

### Sub-Processes

#### 3.1 Add to Cart

- **Input:** Product selection from Customer
- **Process:** Validate product availability and add to cart
- **Output:** Cart update confirmation
- **Data Store Interaction:** Insert/Update cart record in **D3: Cart & Order Database**

#### 3.2 View Cart

- **Input:** Cart view request from Customer
- **Process:** Retrieve all items from cart
- **Output:** Cart details (products, quantity, total price)
- **Data Store Interaction:** Read from **D3: Cart & Order Database**

#### 3.3 Place Order

- **Input:** Order request (checkout) from Customer
- **Process:** Validate cart items, generate order ID, store order
- **Output:** Order confirmation



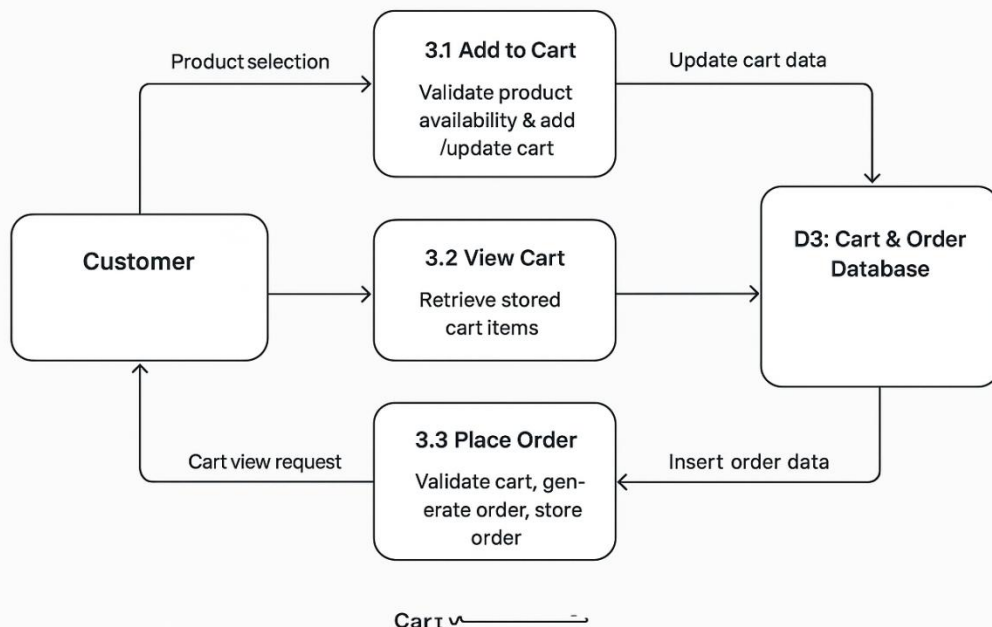
- **Data Store Interaction:** Insert order details into **D3: Cart & Order**

## Database

## Data Flows

| From                                        | To                       | Data          |
|---------------------------------------------|--------------------------|---------------|
| Customer → 3.1 Add to Cart                  | Product selection        | Input         |
| 3.1 Add to Cart → D3: Cart & Order Database | Update cart data         | Insert/Update |
| 3.1 Add to Cart → Customer                  | Cart update confirmation | Output        |
| Customer → 3.2 View Cart                    | Cart view request        | Input         |
| 3.2 View Cart → D3: Cart & Order Database   | Fetch cart items         | Query         |
| 3.2 View Cart → Customer                    | Cart details             | Output        |
| Customer → 3.3 Place Order                  | Place order request      | Input         |
| 3.3 Place Order → D3: Cart & Order Database | Insert order data        | Insert        |
| 3.3 Place Order → Customer                  | Order confirmation       | Output        |

### DFD Level-2 – 3.0 Cart & Order Management – eAuto Management



## **4.0 Payment Processing**

### **External Entities**

- Customer
- Bank / UPI Gateway

### **Data Store**

- D4: Payment Records

### **Sub-Processes**

#### **4.1 Initiate Payment**

- **Input:** Payment details (card/UPI/net banking/cash request) from Customer
- **Process:** Validate input format and forward request
- **Output:** Payment initiation status
- **Data Flow:** Send payment details to Bank/UPI Gateway

#### **4.2 Verify Payment Details**

- **Input:** Payment validation request from 4.1
- **Process:** Bank/UPI Gateway checks account/card details
- **Output:** Valid/Invalid response
- **Data Flow:** Response sent back to 4.2 Verify Payment Details

#### **4.3 Confirm Transaction**

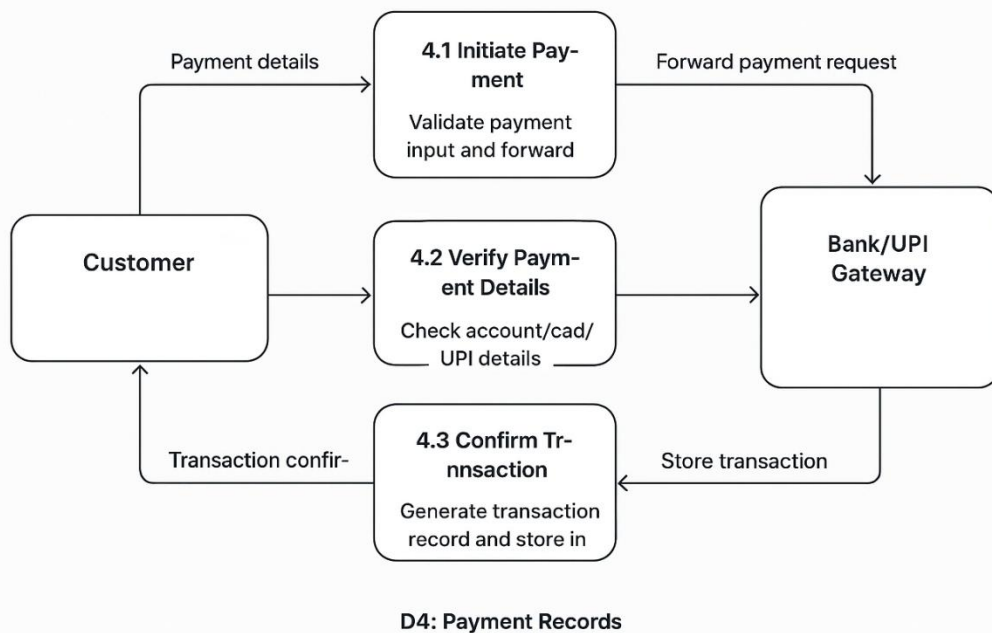
- **Input:** Valid payment confirmation from Bank/UPI Gateway
- **Process:** Generate transaction record and store in DB
- **Output:** Transaction confirmation / Failure message

- **Data Store Interaction: Insert into D4: Payment Records**

### Data Flows

| From                                                 | To                       | Data     |
|------------------------------------------------------|--------------------------|----------|
| Customer → 4.1 Initiate Payment                      | Payment details          | Input    |
| 4.1 Initiate Payment → Bank/UPI Gateway              | Forward payment request  | Request  |
| Bank/UPI Gateway → 4.2 Verify Payment Details        | Verification response    | Response |
| 4.2 Verify Payment Details → 4.3 Confirm Transaction | Validated result         | Input    |
| 4.3 Confirm Transaction → D4: Payment Records        | Store transaction        | Insert   |
| 4.3 Confirm Transaction → Customer                   | Transaction confirmation | Output   |

### DFD Level-2 – 4.0 Payment Processing – eAuto Management System



## 5.0 Supplier & Inventory Management

### External Entities

- Supplier
- Admin

### Data Store

- D5: Inventory Database

### Sub-Processes

#### 5.1 Update Stock

- **Input:** Stock info update from Supplier/Admin
- **Process:** Validate stock details and update inventory records
- **Output:** Stock update confirmation
- **Data Store Interaction:** Update **D5: Inventory Database**

#### 5.2 Check Inventory

- **Input:** Inventory check request from Admin/System
- **Process:** Retrieve current stock details
- **Output:** Stock availability report
- **Data Store Interaction:** Read from **D5: Inventory Database**

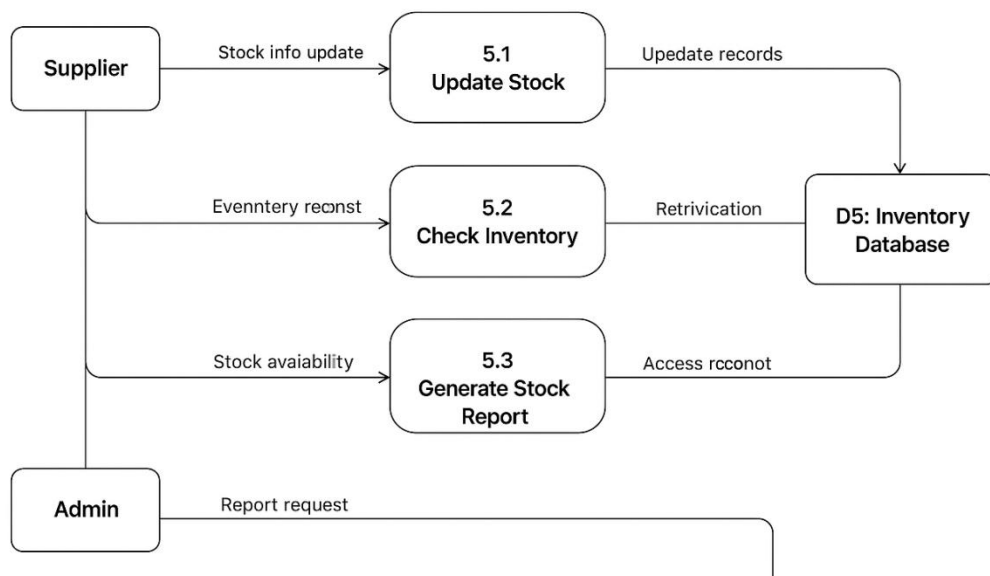
#### 5.3 Generate Stock Report

- **Input:** Report request from Admin
- **Process:** Compile stock levels, shortages, and supplier info
- **Output:** Inventory report
- **Data Store Interaction:** Access data from **D5: Inventory Database**

## Data Flows

| From                                               | To                        | Data   |
|----------------------------------------------------|---------------------------|--------|
| Supplier/Admin → 5.1 Update Stock                  | Stock info update         | Input  |
| 5.1 Update Stock → D5: Inventory Database          | Update records            | Update |
| 5.1 Update Stock → Supplier/Admin                  | Stock confirmation        | Output |
| Admin/System → 5.2 Check Inventory                 | Inventory check request   | Input  |
| 5.2 Check Inventory → D5: Inventory Database       | Retrieve stock data       | Query  |
| 5.2 Check Inventory → Admin/System                 | Stock availability report | Output |
| Admin → 5.3 Generate Stock Report                  | Report request            | Input  |
| 5.3 Generate Stock Report → D5: Inventory Database | Access data               | Query  |
| 5.3 Generate Stock Report → Admin                  | Inventory report          | Output |

### DFD Level-2 – 5.0 Supplier & Inventory Management – eAuto Management System



## 6.0 Delivery Management

### External Entities

- System
- Delivery Staff
- Customer

### Data Store

- D6: Delivery Records

### Sub-Processes

#### 6.1 Assign Delivery

- **Input:** Delivery assignment request from System
- **Process:** Assign order to available Delivery Staff
- **Output:** Delivery assignment confirmation
- **Data Store Interaction:** Insert assignment into **D6: Delivery Records**

#### 6.2 Update Delivery Status

- **Input:** Delivery progress/status from Delivery Staff
- **Process:** Record status updates (Out for delivery, Delivered, Failed)
- **Output:** Updated delivery status
- **Data Store Interaction:** Update **D6: Delivery Records**

#### 6.3 Confirm Delivery

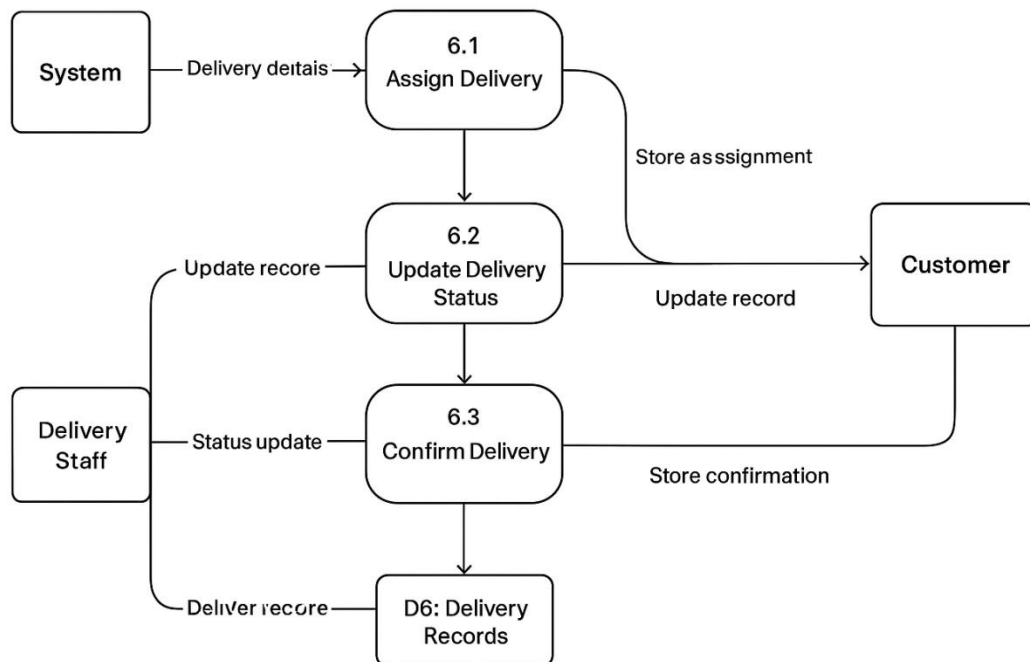
- **Input:** Delivery completion info from Delivery Staff
- **Process:** Validate and confirm successful delivery
- **Output:** Delivery confirmation sent to Customer & System

- **Data Store Interaction:** Store confirmation in **D6: Delivery Records**

### Data Flows

| From                                              | To                          | Data   |
|---------------------------------------------------|-----------------------------|--------|
| System → 6.1 Assign Delivery                      | Delivery assignment request | Input  |
| 6.1 Assign Delivery → D6: Delivery Records        | Store assignment            | Insert |
| 6.1 Assign Delivery → Delivery Staff              | Assignment confirmation     | Output |
| Delivery Staff → 6.2 Update Delivery Status       | Status update               | Input  |
| 6.2 Update Delivery Status → D6: Delivery Records | Update record               | Update |
| 6.2 Update Delivery Status → System/Customer      | Updated status              | Output |
| Delivery Staff → 6.3 Confirm Delivery             | Delivery completion info    | Input  |
| 6.3 Confirm Delivery → D6: Delivery Records       | Store confirmation          | Insert |
| 6.3 Confirm Delivery → Customer                   | Delivery confirmation       | Output |
| 6.3 Confirm Delivery → System                     | Delivery confirmation       | Output |

### DFD Level-2 – 6.0 Delivery Management – eAuto Management System



## 7.0 Reports & Analytics

### External Entities

- Admin
- System

### Data Store

- D7: Reports Database

### Sub-Processes

#### 7.1 Generate Sales Report

- **Input:** Report request (sales) from Admin
- **Process:** Compile order and payment data to generate sales insights
- **Output:** Sales report
- **Data Store Interaction:** Retrieve/store in **D7: Reports Database**

#### 7.2 Generate Stock Report

- **Input:** Report request (stock) from Admin
- **Process:** Collect inventory data, show shortages and availability
- **Output:** Stock report
- **Data Store Interaction:** Retrieve/store in **D7: Reports Database**

#### 7.3 Generate Delivery Report

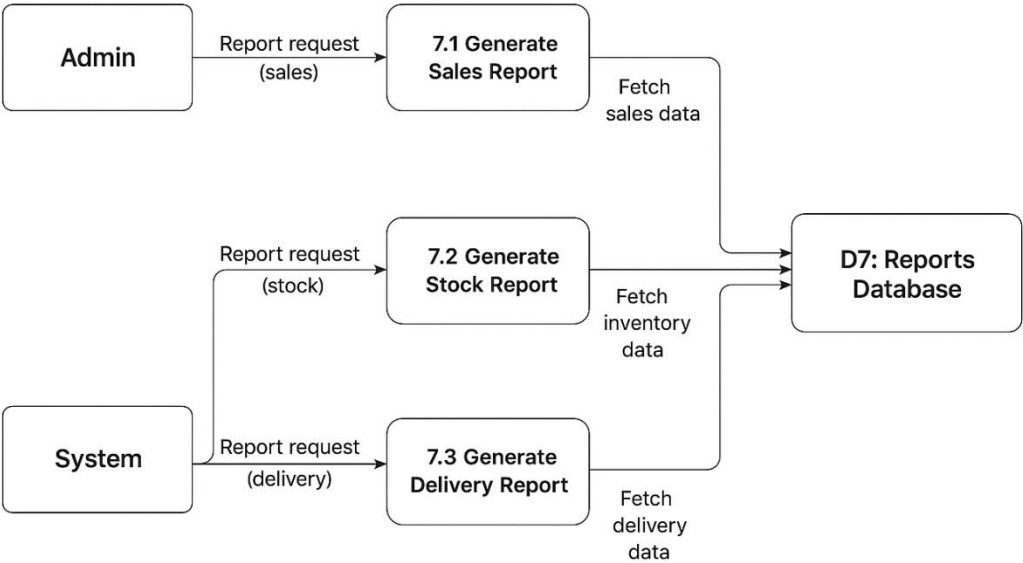
- **Input:** Report request (delivery) from Admin
- **Process:** Summarize delivery records, delays, completion rates
- **Output:** Delivery report
- **Data Store Interaction:** Retrieve/store in **D7: Reports Database**



Data Flows

| From                                                | To                                      | Data        |
|-----------------------------------------------------|-----------------------------------------|-------------|
| Admin → 7.1 / 7.2 / 7.3                             | Report request                          | Input       |
| 7.1 Generate Sales Report → D7: Reports Database    | Fetch sales data                        | Query/Store |
| 7.1 Generate Sales Report → Admin                   | Sales report                            | Output      |
| 7.2 Generate Stock Report → D7: Reports Database    | Fetch inventory data                    | Query/Store |
| 7.2 Generate Stock Report → Admin                   | Stock report                            | Output      |
| 7.3 Generate Delivery Report → D7: Reports Database | Fetch delivery data                     | Query/Store |
| 7.3 Generate Delivery Report → Admin                | Delivery report                         | Output      |
| System → D7: Reports Database                       | Data sync (orders, inventory, delivery) | Update      |

DFD Level-2 - 7.0 Reports & Analytics - eAuto Management System



## 6.3 Explanation of Data Stores (MongoDB Collections)

### D1: Users Collection

Stores details of registered customers and admins.

```
{  
  "user_id": 1,  
  "fullname": "John Doe",  
  "email": "john@example.com",  
  "password": "hashed_password",  
  "contact": "9876543210",  
  "role": "customer" // can be 'customer' or 'admin'  
}
```

---

### D2: Products Collection

Stores spare parts/product details.

```
{  
  "product_id": 101,  
  "name": "Hero Splendor Brake Shoe",  
  "brand": "Hero",  
  "category": "Brakes",  
  "price": 450,  
  "stock": 20,  
  "discount": 10,
```

```
"description": "High-quality brake shoe for Hero Splendor bike",  
"image_url": "uploads/products/brake-shoe.jpg" }
```

### **D3: Cart Collection**

Maintains user's selected items before checkout.

```
{  
  "cart_id": 1,  
  "user_id": 1,  
  "products": [  
    { "product_id": 101, "quantity": 2 },  
    { "product_id": 105, "quantity": 1 }  
  ],  
  "updated_at": "2025-08-26T12:00:00Z"  
}
```

---

### **D4: Orders Collection**

Stores confirmed orders with cart items and order status.

```
{  
  "order_id": 5001,  
  "user_id": 1,  
  "cart_items": [  
    { "product_id": 101, "quantity": 2, "price": 450 },  
    { "product_id": 105, "quantity": 1, "price": 1200 }  
  ]  
}
```

```
],  
  
"total_amount": 2100,  
  
"status": "Pending", // Pending, Shipped, Delivered, Cancelled  
  
"created_at": "2025-08-26T13:15:00Z"  
  
}
```

### **D5: Payments Collection**

Stores payment transaction details for orders.

```
{  
  
"payment_id": "P123",  
  
"order_id": 5001,  
  
"method": "UPI", // UPI, Card, NetBanking, Wallet  
  
"amount": 2100,  
  
"status": "Success", // Success, Failed, Pending  
  
"transaction_date": "2025-08-26T13:20:00Z"  
  
}
```

---

### **D6: Delivery Records Collection**

Tracks delivery status of orders.

```
{  
  
"delivery_id": 2001,  
  
"order_id": 5001,  
  
"delivery_staff": "Ravi",
```

```
"delivery_address": "123 Main Street, Surat",  
"status": "Delivered", // Assigned, Out for Delivery, Delivered  
"delivered_at": "2025-08-27T17:00:00Z"  
}
```

---

## **D7: Reports Collection**

Stores analytics and report data generated for admins.

```
{  
  "report_id": 301,  
  "type": "Sales",  
  "generated_at": "2025-08-26T18:30:00Z",  
  "data": {  
    "total_sales": 120000,  
    "top_products": ["Hero Splendor Brake Shoe", "Bajaj Clutch Plate"]  
  }  
}
```

---

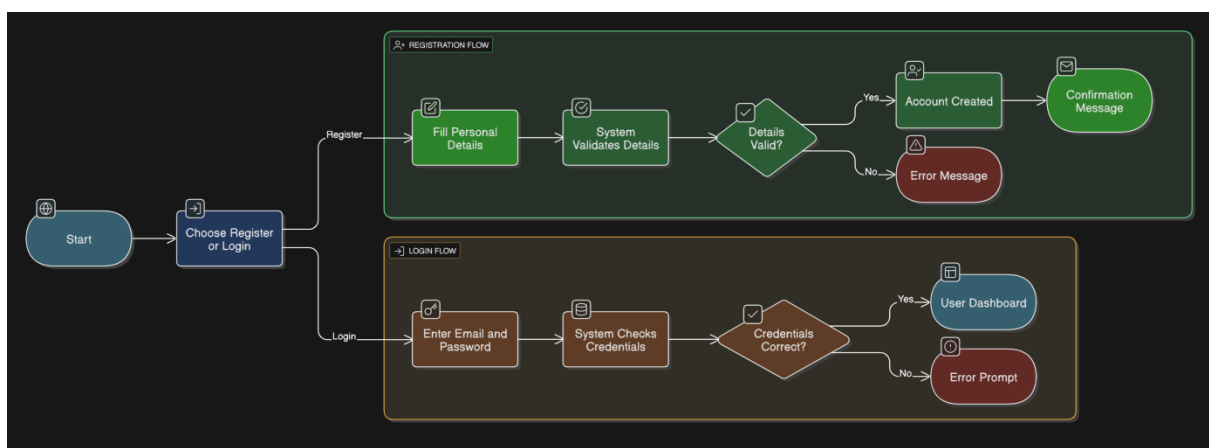
## 7. Flow Chart

### 7.1 User Registration & Login Flow

#### Process Steps:

1. User visits the website.
2. Chooses **Register** or **Login** option.
3. For **Registration**:
  - Fill personal details (name, email, password, contact).
  - System validates details.
  - If valid → Account created → Confirmation message.
  - If invalid → Error message.
4. For **Login**:
  - Enter email & password.
  - System checks credentials from DB.
  - If correct → User dashboard.
  - If incorrect → Error prompt.

#### Flow Chart Structure (Textual):

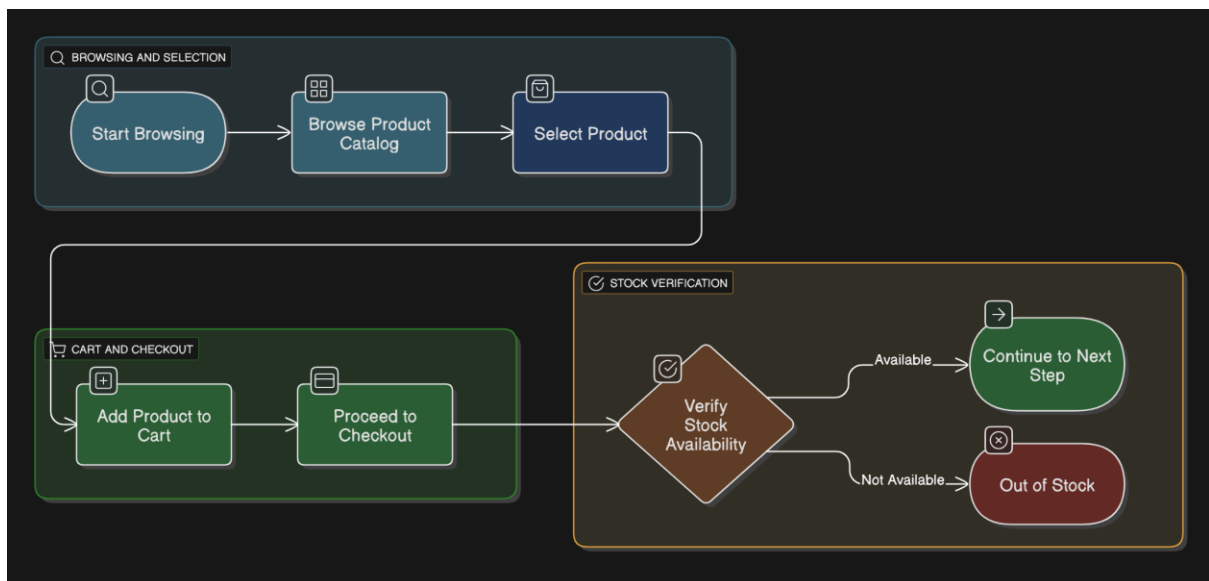


## 7.2 Product Purchase Flow

### Process Steps:

1. User browses product catalog.
2. Selects a product.
3. Adds product to cart.
4. Proceeds to checkout.
5. System verifies stock availability.
6. If available → Continue.
7. If not available → Show “Out of Stock.”

### Flow Chart Structure (Textual):

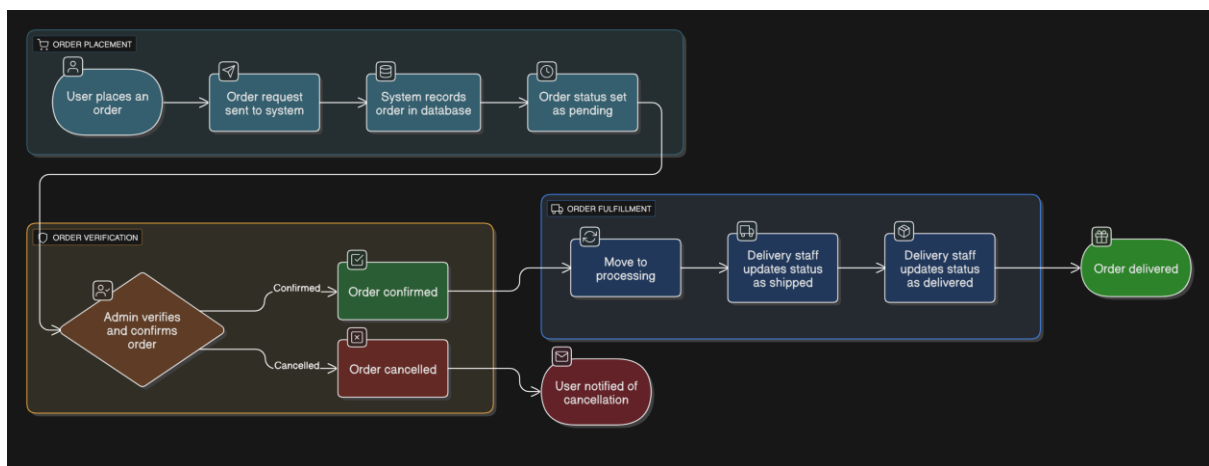


## 7.3 Order Management Flow

### Process Steps:

1. User places an order.
2. Order request sent to system.
3. System records order in DB.
4. Order status set as **Pending**.
5. Admin verifies and confirms order.
6. If confirmed → Move to **Processing**.
7. If cancelled → Notify user.
8. Delivery staff updates status as **Shipped** → **Delivered**.

### Flow Chart Structure (Textual):



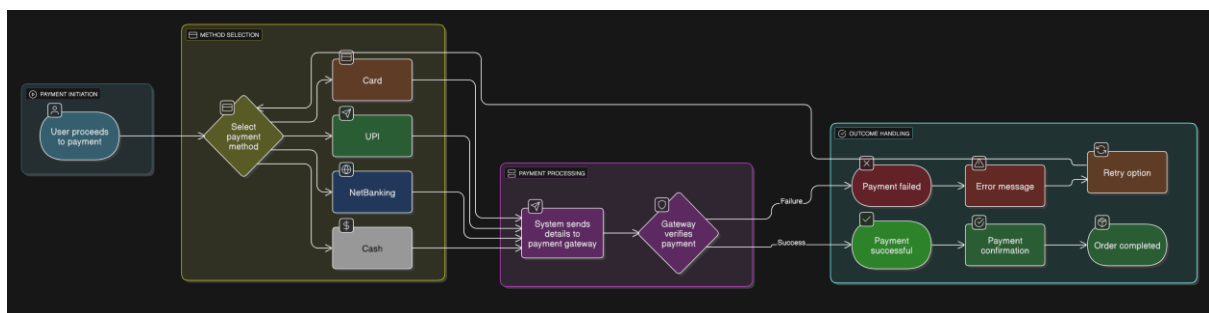


## 7.4 Payment Processing Flow

### Process Steps:

1. User proceeds to payment.
2. Selects payment method (Card/UPI/NetBanking/Cash).
3. System sends details to payment gateway.
4. Gateway verifies payment.
5. If successful → Payment Confirmation → Order Completed.
6. If failed → Error Message → Retry option.

### Flow Chart Structure (Textual):



## **8. Implementation Details**

### **8.1 Frontend (React JS, Bootstrap, AOS)**

- The frontend of the system is developed using **React JS**, ensuring a modular and reusable component-based structure.
  - **Bootstrap 5** is used for responsive layouts, grids, and pre-styled UI components like buttons, modals, and navigation.
  - **AOS (Animate On Scroll)** library is integrated to enhance the user experience with smooth animations during scrolling.
  - Major pages/components include:
    - Home.js → Displays featured products and categories.
    - Product Pages (Hero.js, Honda.js, Bajaj.js) → Brand-wise product listings.
    - Cart.js → Displays cart items dynamically.
    - Checkout.js → Checkout form with user details and payment selection.
    - Profile.js, Orders.js, DeliveryAddress.js → User account management.
  - The frontend communicates with backend APIs using **Axios (fetching JSON data)**.
-

## 8.2 Backend (PHP – Core PHP for APIs & Logic)

- The backend is built using **Core PHP** for server-side processing and API creation.
  - PHP scripts are structured into controllers and service layers for better maintainability.
  - APIs follow **REST principles**, handling GET, POST, PUT, and DELETE requests.
  - Core features handled in backend:
    - User authentication (Login/Register).
    - Product management (CRUD operations).
    - Cart operations (add, update, remove items).
    - Order placement and status tracking.
    - Payment integration (mock/demo for UPI, Card, Cash on Delivery).
    - Admin APIs for product, user, and order management.
- 

## 8.3 Database (MongoDB Compass – Collections & Schema)

- Database used: **MongoDB**, managed via **MongoDB Compass**.
- Collections:
  - Users → Stores user info (user\_id, name, email, password hash, contact).

- Products → Product details (product\_id, name, brand, price, stock, discount).
- Cart → User cart data with product references and quantities.
- Orders → Order records (order\_id, user\_id, items, status, total amount).
- Payments → Payment transactions (payment\_id, order\_id, method, status).
- DeliveryRecords → Tracks delivery staff assignment and status.
- Reports → Sales and analytics reports for admin.

---

#### 8.4 APIs & Endpoints (PHP → React Communication with JSON)

- APIs are written in PHP and return responses in **JSON format** for React consumption.
- Example Endpoints:
  - **Authentication**
    - POST /api/register.php → Registers a new user.
    - POST /api/login.php → Authenticates user, returns session token.
  - **Products**
    - GET /api/products.php?brand=Hero → Fetch Hero brand products.

- GET /api/product.php?id=101 → Fetch single product details.
  - **Cart**
    - POST /api/cart/add.php → Add product to cart.
    - GET /api/cart/view.php?user\_id=1 → View cart items.
  - **Orders**
    - POST /api/orders/create.php → Place new order.
    - GET /api/orders/view.php?user\_id=1 → View order history.
  - **Payments**
    - POST /api/payments/process.php → Process payment.
- 

## 8.5 Security Features (Authentication, Session Management, Validation)

- **Authentication**
  - Secure login using hashed passwords (bcrypt/MD5).
  - Session tokens issued on successful login.
- **Session Management**
  - PHP sessions maintain user state across requests.
  - Token-based authentication for React frontend API calls.
- **Validation**
  - Client-side (React) form validation for inputs.
  - Server-side (PHP) validation for email, password, and payment details.

- **Data Security**

- Passwords stored in encrypted format.
- APIs restricted with role-based access (User/Admin).

- **Other Features**

- Prevention of SQL Injection (Prepared Statements).
  - Prevention of Cross-Site Scripting (XSS) using input sanitization.
-

## 9. Testing

### 9.1 Test Cases (Unit & Integration)

| Test Case ID | Module               | Test Scenario                   | Input                                                                        | Expected Output                              | Result |
|--------------|----------------------|---------------------------------|------------------------------------------------------------------------------|----------------------------------------------|--------|
| TC-01        | Login & Registration | User enters valid credentials   | Email: <a href="mailto:user@test.com">user@test.com</a> ,<br>Password: 1234  | Redirect to dashboard/homepage               | Pass   |
| TC-02        | Login & Registration | User enters invalid credentials | Email: <a href="mailto:wrong@test.com">wrong@test.com</a> ,<br>Password: xyz | Show error message<br>“Invalid credentials”  | Pass   |
| TC-03        | Cart Management      | Add product to cart             | Product ID: 101,<br>Qty: 2                                                   | Cart updated with correct product & quantity | Pass   |
| TC-04        | Cart Management      | Remove product from cart        | Remove Product<br>ID: 101                                                    | Cart updated with product removed            | Pass   |
| TC-05        | Order Placement      | Place order with valid cart     | Cart → Checkout<br>→ Confirm                                                 | Order ID generated,<br>status = “Pending”    | Pass   |

| Test Case ID | Module             | Test Scenario              | Input                   | Expected Output                                | Result |
|--------------|--------------------|----------------------------|-------------------------|------------------------------------------------|--------|
|              |                    | items                      |                         |                                                |        |
| TC-06        | Payment Processing | Valid payment via UPI/Card | Payment details entered | Payment Success message, status = "Paid"       | Pass   |
| TC-07        | Payment Processing | Invalid payment details    | Wrong Card Number       | Show error message "Transaction Failed"        | Pass   |
| TC-08        | Admin Management   | Add new product            | Product details form    | Product stored in DB & visible in product list | Pass   |
| TC-09        | Admin Management   | Generate Sales Report      | Click "Generate Report" | Report file created with correct data          | Pass   |

## 9.2 User Testing (UI/UX & Functional)

User testing was carried out with a group of **10 students and faculty members**.

The testing focused on **UI clarity, navigation, responsiveness, and functional workflows**.

- ☐ **Login/Register:** Smooth and easy to use.
- ☐ **Product Browsing:** Search and filter features worked correctly.
- ☐ **Cart & Checkout:** Easy navigation from cart to payment.



- ☐ **Responsiveness:** Website adapted well to desktop and mobile screens.
- ☐ **Animations (AOS):** Added better user experience without slowing down performance.

#### Feedback:

- Minor improvements suggested in **search filtering speed** and **dark mode option**.
- 

### 9.3 Admin Testing

Admin panel functionalities were tested for **CRUD operations, report generation, and order management**.

- ☐ Admin successfully added, updated, and deleted products.
  - ☐ Order management system allowed status update (Pending → Shipped → Delivered).
  - ☐ Sales & Inventory Reports generated correctly.
  - ☐ User management (block/unblock) worked as expected.
- 

### 9.4 Performance & Load Testing

Performance testing was performed using **JMeter & manual load simulation**.

| Test Type        | Scenario                         | Result                          |
|------------------|----------------------------------|---------------------------------|
| Page Load Speed  | Homepage load under 2 seconds    | <input type="checkbox"/> Passed |
| Concurrent Users | 50 users browsing simultaneously | <input type="checkbox"/> Passed |

| Test Type                | Scenario                             | Result                          |
|--------------------------|--------------------------------------|---------------------------------|
| Order Placement Stress   | 20 users placing order at same time  | <input type="checkbox"/> Passed |
| Database Query Speed     | Fetching 1000+ products with filters | <input type="checkbox"/> < 1.5s |
| Payment Gateway Response | Average response time < 3s           | <input type="checkbox"/> Passed |

---

## **10. Limitations of the Project**

### **10.1 Technical Limitations**

- The integration between **React (Frontend)** and **PHP (Backend APIs)** can sometimes cause synchronization issues, especially with asynchronous data fetching.
  - **MongoDB Compass** is used as the primary database, which may pose challenges in handling complex relational queries compared to relational databases like MySQL.
  - Security measures such as **advanced encryption, JWT tokens, and OAuth2.0** are not fully implemented, which may limit the robustness of authentication.
  - The system currently relies on a **local server/XAMPP setup** during development, which restricts scalability and performance compared to cloud deployment.
- 

### **10.2 Functional Limitations**

- Payment methods are limited to **basic transactions (Cash, UPI, Net Banking, Debit/Credit Card)** and do not yet support **wallets, EMI, or international gateways**.

- The **delivery management system** is simplified and lacks real-time tracking or integration with third-party logistics providers.
- Reporting and analytics features are basic and do not provide **deep insights or predictive analytics** for sales, user behavior, or inventory trends.
- No dedicated **mobile app** is currently available, restricting accessibility to desktop and web browsers only.

### 10.3 Future Enhancements

To overcome the above limitations and improve the system, the following enhancements can be considered in future iterations:

- Develop a **mobile application (Android & iOS)** for wider accessibility.
  - Implement **AI-driven product recommendations** based on user behavior and purchase history.
  - Introduce **advanced reporting and dashboards** for admins, including sales forecasting, inventory optimization, and customer insights.
  - Enable integration with **third-party delivery services (e.g., Delhivery, Shiprocket)** to provide real-time tracking.
  - Expand **payment gateway support** to include PayPal, Razorpay, wallets, and international payment methods.
  - Deploy the system on **cloud platforms (AWS, Azure, or GCP)** for better scalability, security, and performance.
-

## **11. Conclusion**

### **11.1 Summary of Work**

The project “**eAuto – Bike Spare Parts E-Commerce Platform**” was designed and developed to provide a seamless digital marketplace for two-wheeler spare parts.

The system integrates:

- **Frontend:** React JS, Bootstrap, and AOS for a responsive and dynamic user interface.
- **Backend:** Core PHP for APIs and business logic.
- **Database:** MongoDB Compass for handling product, user, order, and transaction records.

The platform supports **user registration, login, product browsing, cart management, order placement, and payment processing**, along with **admin functionalities** such as managing products, users, inventory, orders, and reports.

This project successfully bridges the gap between suppliers, customers, and delivery systems, offering a reliable solution for bike spare parts e-commerce.

---

### **11.2 Achievements**

- Developed a **fully functional e-commerce system** with **user-side and admin-side modules**.

- Implemented **secure authentication and session management** for protecting user accounts.
- Integrated **cart, checkout, and order management workflows** in real-time.
- Designed a **MongoDB database schema** to handle collections such as users, products, orders, payments, and deliveries.
- Provided **basic reporting and analytics** for admins to monitor sales, inventory, and performance.
- Delivered a **responsive and user-friendly interface** with animations for better UX.

### 11.3 Future Scope

The project lays the foundation for a scalable and advanced platform. In the future, it can be enhanced with:

- **Mobile applications (Android/iOS)** for better accessibility.
- **AI-driven product recommendations** to personalize the shopping experience.
- **Advanced reporting & dashboards** with predictive analytics for business intelligence.
- **Integration with third-party delivery partners** for real-time shipment tracking.
- **Expanded payment options**, including PayPal, wallets, EMI, and international gateways.

- **Cloud deployment** (AWS, Azure, GCP) for improved scalability, performance, and global reach.
-

## **12. References**

1. **React JS Documentation** – <https://react.dev>
  2. **Bootstrap 5 Documentation** – <https://getbootstrap.com>
  3. **MongoDB Documentation** – <https://www.mongodb.com/docs>
  4. **PHP Official Documentation** – <https://www.php.net/docs.php>
  5. **MDN Web Docs (HTML, CSS, JavaScript)** – <https://developer.mozilla.org>
  6. **W3Schools Tutorials** – <https://www.w3schools.com>
  7. **GeeksforGeeks (Web Development & Database Concepts)** – <https://www.geeksforgeeks.org>
  8. **TutorialsPoint** – <https://www.tutorialspoint.com>
  9. **Stack Overflow** – <https://stackoverflow.com> (for troubleshooting and coding references)
  10. **Research papers and articles on E-Commerce Platforms, Payment Systems, and Inventory Management** accessed via Google Scholar.
-



